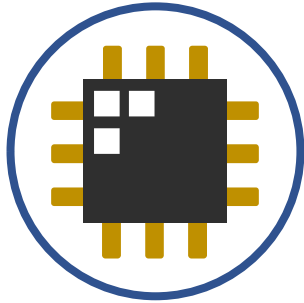




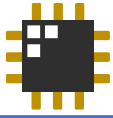
EEE 113:

Introduction to Electrical and Electronics Engineering Systems

Week 3 – Introduction to Assembly Architecting a Processor

1

For this lecture, we will be covering an introduction to assembly language, which is the low-level programming language of a computer. Given the assembly language, we will learn to convert it to the equivalent machine code, which is just a string of 0s and 1s that controls the different logic blocks inside the computer. From there, we will start architecting a computer processor given the instructions that we want it to do. This will be an exciting lecture since we will be building a processor given all of the knowledge that we have accumulated so far, from logic gates, to logic blocks (adders, multiplexers, decoders), and finally to registers.



Talking to your computer: Assembly language

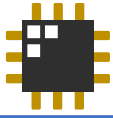
- Before we start creating a processor, we should know its *language* first
- Computers use a low-level language called **assembly**.
- Assembly contains **mnemonic codes (instructions)** that specify what the processor should do.
- An instruction specifies the **operation** to perform and the **operands** on which to operate

ADD R1, R0, 1

(R1 = R0 + 1)

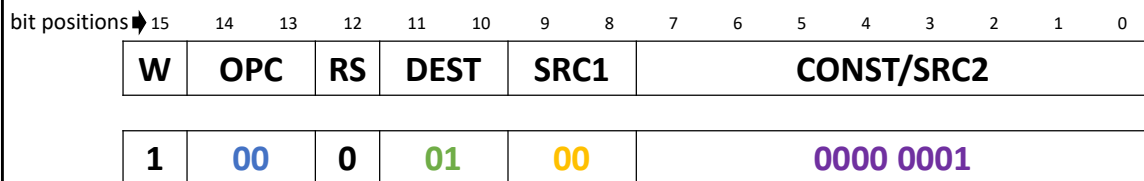
2

Before we start architecting a processor, we should understand its language first. Assembly language is a low-level programming language that helps to communicate directly with computer hardware. It uses mnemonic codes, also called instructions, that represent the operations that a processor has to do. Shown on the slide is a sample instruction. It simply tells the processor to add the contents of register R0 to 1 and then store the result to register R1, as highlighted in the last line enclosed in parentheses). You can imagine R0 and R1 as variables for now. These 'variables' contain some data and you want to operate on that data. If you're coming from a programming background, $R1 = R0 + 1$ makes sense and you know what you should expect to happen when that instruction evaluates.



Talking to your computer: Assembly language

- Each instruction has an **equivalent binary string**. The processor parses the binary bits that would go to different *control signals*



00 – ADD
 01 – SUB
 This instruction updates the register file
 This instruction's second operand is a constant

ADD R1, R0, 1

(R1 = R0 + 1) Think of Rx as your variable in a program!

As introduced before, these instructions are just mnemonics. They are written like that, with English words like ADD, since it makes programming easy and intuitive. However, computers don't understand the word 'ADD'. Computers operate in the world of 0s and 1s. Thus, these instructions are actually converted into an equivalent binary string that the computer understands. Every instruction will always have an equivalent binary string. This binary string is then parsed by the computer which will then go to different blocks inside the processor to control the different logic blocks to execute what the instruction needs to do. We will see more on how different bits would play a role in the execution of an instruction.

For now, here's the same ADD instruction being translated into its 16-bit binary counterpart. As you can see, every part of the 16-bit string has some meaning and has some translation from the instruction ADD R1, R0, 1. For this specific instruction encoding, we have the following partitioning (starting from left, MSB, to the right, LSB)

The most significant bit (bit position 15) indicates that the instruction would be updating the register file because the operation is $R1 = R0 + 1$, thus R1 needs to be updated.

The next two bits after (bit position 14 and 13) that indicate what the operation

should be, which is addition in this case. Let's say that the 2-bit code for addition is 00.

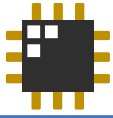
The next bit after that (bit position 12) indicates that the second operand is a constant. The instruction is adding the contents of R0 to the constant 1. The second operand is a constant, so we tell the processor that by putting 0 on that bit.

The next two bits after that (bit position 11 and 10) indicate what the destination register is. For this sample processor that we will be building, we only have 4 different registers, R0, R1, R2, and R3, thus we need two bits to uniquely identify each of them. Since the destination register of the instruction is R1 (R1 will contain the result of $R0 + 1$), then we put 01 on those two bits.

The next two bits after that (bit position 9 and 8) indicate the 1st source register operand. It follows the same mapping earlier, but since our first operand is R0, then we put 00 on those two bits.

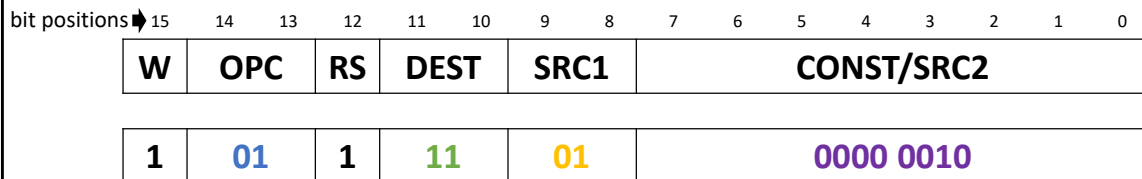
Then, finally, we have the last 8 bits of the instruction (bit position 7 down to 0). These 8 bits will be allotted for the constant term in our instruction. The constant is 1 and writing that in 8-bit binary, we have 0000 0001.

That's it. We have just converted the ADD R1, R0, 1 instruction into its equivalent machine code. This machine code (a string of binary bits) will be used by the processor that we will be designing later to perform the specific operation.



Talking to your computer: Assembly language

- Each instruction has an **equivalent binary string**. The processor parses the binary bits that would go to different *control signals*



00 – ADD
 01 – SUB
 This instruction updates the register file
 This instruction's second operand is a register

SUB R3, R1, R2

(R3 = R1 – R2) Think of Rx as your variable in a program!

Here's another example but using a different instruction, so we can get the hang of it.

This instruction is a subtraction instruction, SUB R3, R1, R2, which is essentially performing $R3 = R1 - R2$.

Again, following the same pattern that we have identified earlier when converting to binary bits, we can start writing what each binary value should be:

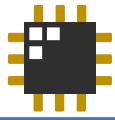
The most significant bit (bit position 15) indicates that the instruction would be updating the register file because the operation is $R3 = R1 - R2$, thus R3 needs to be updated.

The next two bits after that (bit position 14 and 13) indicate what the operation should be, which is subtraction in this case. Let's say that the 2-bit code for subtraction is 01.

The next bit after that indicates that (bit position 12) the second operand is a register. Contrary to the previous instruction this utilizes a register as a second operand. We put that distinction by putting 1 on this bit.

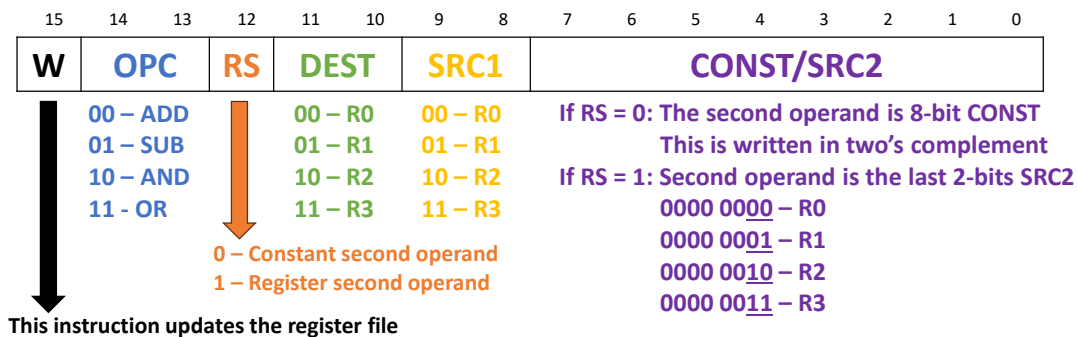
The next two bits after that (bit position 11 and 10) indicate what the destination register is. The instruction says that R3 should contain the result of the subtraction, therefore we put 11 on those two bits.

The next two bits after that (bit position 9 and 8) indicate the 1st source register operand. Our first operand is R1, then we put 01 on those two bits. Then, finally, we have the last 8 bits of the instruction (bit position 7 down to 0). Since we noted earlier that our second operand is a register, we actually only need 2 bits to represent the register number. Thus, we will just put 0s on the upper 6 bits and then write the 10, corresponding to R2, on the last two bits. Thus, when writing the entire 8-bits in binary, we have 0000 0010. And that's it, that would be the equivalent machine code or binary string for the instruction SUB R3, R1, R2.



Talking to your computer: Assembly language

- Each instruction has an **equivalent binary string**. The processor parses the binary bits that would go to different *control signals*



Note: For EEE 113, we will be architecting a custom processor for easy learning ☺
The binary encoding for that custom processor is shown here.

5

Here's a summary of the instruction encoding that we have used earlier. This determines how an instruction is converted into a binary string that the processor understands.

Bit position 15 (the MSB), named W for 'write enable', identifies whether the instruction writes to the register file. All of the instructions we have so far write to the register file because it needs to update the destination register whenever an operation is performed. Other instructions will be presented later that do not write to the register file. We will explore such instructions later, but for now, this bit remains 1.

Bit position 14 and 13, named OPC for 'operation code', identifies the operation that needs to be performed. Since we have 2 bits, we can do up to four different operations. For this sample processor that we are analyzing, we have 00 for addition, 01 for subtraction, 10 for bitwise (bit by bit) AND operation, and 11 for bitwise OR operation.

Bit position 12, named RS for 'register select' identifies whether the operation uses a constant or a register for the second operand. The previous two instructions we had earlier used this bit (ADD R1, R0, 1 uses a constant as second operand but SUB R3, R1, R2 uses a register as a second operand). This will be relevant for our interpretation of the last 8 bits of the instruction

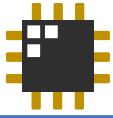
(CONST/SRC2).

Bit position 11 and 10, named DEST for 'destination register', identifies the destination register for our operations. So far, all our instructions will write to some register, so this field is important to identify which one to write. Since we have 2 bits, we have four registers that can be set as the destination. The 2-bit mapping is simply the binary equivalent of the register name: 00 for R0, 01 for R1, 10 for R2, and 11 for R3.

Bit position 9 and 8, named SRC1 for 'source register 1', identifies the first operand register. All our instructions always have a register as the first operand. This follows the same 2-bit mapping for the register names.

Bit position 7 down to 0 (8 bits), named CONST/SRC2, has two purposes. It is the 8-bit constant, CONST, if RS (bit 12) is 0. This is the two's complement equivalent of whatever the constant in our instruction is, allowing for negative number representations. If RS (bit 12) is 1 however, indicating that the second operand is a register and not a constant, only the last 2 bits are relevant, which is the 2-bit mapping for the register names. The upper 6 bits remain 0 in this case.

We will be updating this instruction encoding later in this lecture as we support new instructions for the processor. For now, this is the current encoding that we will be looking at.



Playing around with instructions

- If we have the following instructions, what would be the contents of the register file every step of the way? (Assume all register values start at 0)

ADDI R3, R0, 113

ADDI R2, R2, 21

SUB R1, R3, R2

ANDI R1, R1, 15

OR R0, R2, R1

Note: We are changing the syntax a bit for easier identification:

ADDI/SUBI/ANDI/ORI* means RS = 0 (the second operand is a constant)

ADD/SUB/AND/OR means RS = 1 (the second operand is a register)

Additional note on AND and OR:

These are bitwise operations.

You'll have to write down the binary bits to be able to perform these operations

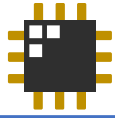
*The additional I at the end means *immediate*, computer architecture term for a constant

6

Let's have some practice so that we become comfortable writing instructions. Assembly programming uses the same programming mindset that you use for high-level languages like Python or C. However, assembly has very limited functionality per instruction. You have to break down a problem into parts. Thus, assembly programs are almost always longer when doing the same function implemented in a high-level language. That's how computers really work!

In this example, we have four instructions, we want to identify what the register values would be as every instruction finishes executing.

A small note, by the way, is that we are formatting the instructions that use a constant with an 'I' suffix (for example, ADDI/SUBI/ANDI/ORI) to differentiate whether the instruction uses a constant or a register as the second operand. Instructions that have a register as the second operand will retain the original names without the suffix (ADD/SUB/AND/OR)



Playing around with instructions

- If we have the following instructions, what would be the contents of the register file every step of the way? (Assume all register values start at 0)

ADDI R3, R0, 113

R3 = R0 (initially 0) + 113 = 113

ADDI R2, R2, 21

R2 = R2 (initially 0) + 21 = 21

SUB R1, R3, R2

R1 = R3 (113) – R2 (21) = 92

ANDI R1, R1, 15

R1 = R1 (92) bitwise AND 15 = 12

OR R0, R2, R1

R0 = R2 (21) bitwise OR R1 (12) = 29

Note: We are changing the syntax a bit for easier identification:

ADDI/SUBI/ANDI/ORI* means RS = 0 (the second operand is a constant)

ADD/SUB/AND/OR means RS = 1 (the second operand is a register)

Additional note on AND and OR:

These are bitwise operations.

You'll have to write down the binary bits to be able to perform these operations

*The additional I at the end means *immediate*, computer architecture term for a constant

Running the instructions line by line, we have:

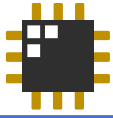
ADDI R3, R0, 113: If R0 is initially 0, then we perform $0 + 113$ (a constant in the instruction) = 113. Therefore, $R3 = 113$.

ADDI R2, R2, 21: If R2 is initially 0, then we perform $0 + 21$ (a constant in the instruction) = 21. Therefore, $R2 = 21$.

SUB R1, R3, R2: Since $R3 = 113$ and $R2 = 21$, then we perform $R3 - R2 = 113 - 21 = 92$. Therefore, $R1 = 92$.

ANDI R1, R1, 15: Since $R1 = 92$, we perform 92 bitwise ANDed with 15 (a constant in the instruction). In binary, this is 0101 1100 (this is 92) ANDed with 0000 1111 (this is 15) = 0000 1100 (this is 12). Therefore, $R1 = 12$

OR R0, R2, R1: Since $R2 = 21$ and $R1 = 12$, then we perform 21 bitwise ORed with 12. In binary, this is 0001 0101 (this is 21) ORed with 0000 1100 (this is 12) = 0001 1101 (this is 29). Therefore, $R0 = 29$.



Converting instructions to machine code

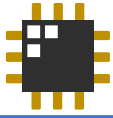
- For us to run an instruction on the processor, we have to convert the instruction into its binary string (machine code) equivalent.
 - There is dedicated software to do this for actual processors. They are called **assemblers**.
- For now, let's practice becoming the assembler. 😊
 - If you know what the different binary fields are for an instruction, we can do it!

W	OPC	RS	DEST	SRC1	CONST/SRC2
↓ This instruction updates the register file	00 – ADD	↓ 0 – Constant second operand 1 – Register second operand	00 – R0	00 – R0	If RS = 0: The second operand is 8-bit CONST This is written in two's complement
	01 – SUB		01 – R1	01 – R1	If RS = 1: Second operand is the last 2-bits SRC2
	10 – AND		10 – R2	10 – R2	0000 0000 – R0
	11 – OR		11 – R3	11 – R3	0000 0001 – R1 0000 0010 – R2 0000 0011 – R3

8

When we started this lecture, the instruction was presented with its corresponding binary string. We were able to match what each of the binary bits corresponded to when looking at the instruction. In real life, this conversion process from an instruction into its machine code (binary string) equivalent is being done by software called assemblers. This is especially important the more instructions we have to translate since it can get a little tedious.

But for now, let's try to be an assembler first to become familiar with the process. Since we have already identified the meaning of the different bits, we can write any instruction (as long as the processor supports it) into its corresponding machine code.



Converting instructions to machine code

- Convert the following instruction into machine code (write the code in hex)

SUBI R3, R1, -31

W	OPC	RS	DEST	SRC1	CONST/SRC2
00 – ADD 01 – SUB 10 – AND 11 – OR	00 – R0 01 – R1 10 – R2 11 – R3	00 – R0 01 – R1 10 – R2 11 – R3	00 – R0 01 – R1 10 – R2 11 – R3	00 – R0 01 – R1 10 – R2 11 – R3	If RS = 0: The second operand is 8-bit CONST This is written in two's complement If RS = 1: Second operand is the last 2-bits SRC2 0000 0000 – R0 0000 0001 – R1 0000 0010 – R2 0000 0011 – R3
This instruction updates the register file					

9

Here's an example, let's try to convert SUBI R3, R1, -31 into its corresponding binary equivalent. This instruction performs $R3 = R1 - (-31)$.

Since this instruction will need to update the value of R3, it has to write to the register file, so $W = 1$.

Since this instruction performs subtractions, $OPC = 01$ according to the mapping.

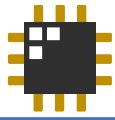
Since this instruction has a constant for its second operand, $RS = 0$.

Since this instruction has R3 as its destination register (the one to update), $DEST = 11$ according to the mapping.

Since this instruction has R1 as the first operand, $SRC1 = 01$ according to the mapping.

Since this instruction has a constant (-31) as the second operand, we will write -31 in 8-bit two's complement notation.

31 is 0001 1111. Since we want the negative representation, we perform the 'two's complement process'. Invert everything plus 1 gives us 1110 0001. This is the two's complement representation of -31.



Converting instructions to machine code

- Convert the following instruction into machine code (write the code in hex)

SUBI R3, R1, -31

W	OPC	RS	DEST	SRC1	CONST/SRC2
00 – ADD 01 – SUB 10 – AND 11 – OR	00 – R0 01 – R1 10 – R2 11 – R3	00 – R0 01 – R1 10 – R2 11 – R3	00 – R0 01 – R1 10 – R2 11 – R3	00 – R0 01 – R1 10 – R2 11 – R3	If RS = 0: The second operand is 8-bit CONST This is written in two's complement If RS = 1: Second operand is the last 2-bits SRC2 0000 0000 – R0 0000 0001 – R1 0000 0010 – R2 0000 0011 – R3
This instruction updates the register file					
1	01	0	11	01	1110 0001

10

Thus, we have the following:

W = 1 (it writes to the register file)

OPC = 01 (SUB)

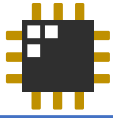
RS = 0 (second operand is a register)

DEST = 11 (R3)

SRC1 = 01 (R1)

CONST = 1110 0001 (-31 in two's complement)

1 01 0 11 01 1110 0001



Converting instructions to machine code

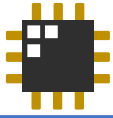
- Machine code for instructions is typically written in hex to minimize typo errors

SUBI R3, R1, -31

1	01	0	11	01	1110 0001
---	----	---	----	----	-----------

11

Typically, the machine code for instructions is written in hex to avoid errors when writing out the long string of binary bits.



Converting instructions to machine code

- Machine code for instructions is typically written in hex to minimize typo errors

SUBI R3, R1, -31

1	01	0	11	01	1110 0001
---	----	---	----	----	-----------

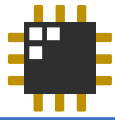
1010 1101 1110 0001

0xADE1

(The equivalent machine code in hex)

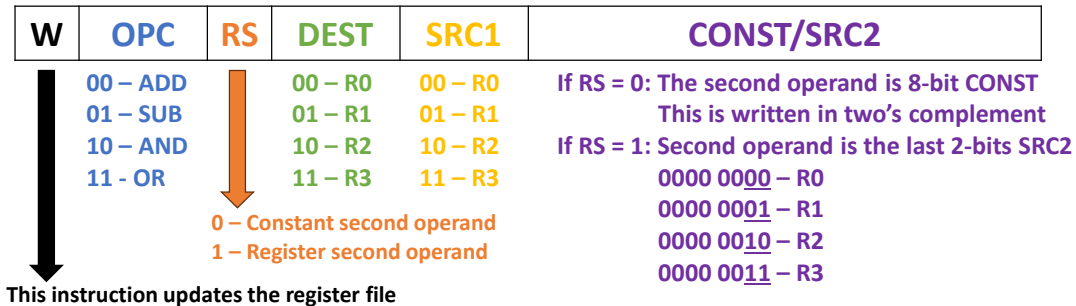
12

The machine code equivalent of SUBI R3, R1, -31 is 0xADE1. 😊



Recap: Instruction encoding

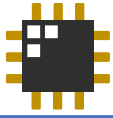
- Each instruction has an **equivalent binary string**. The processor parses the binary bits that would go to different *control signals*



We can now start architecting the processor!

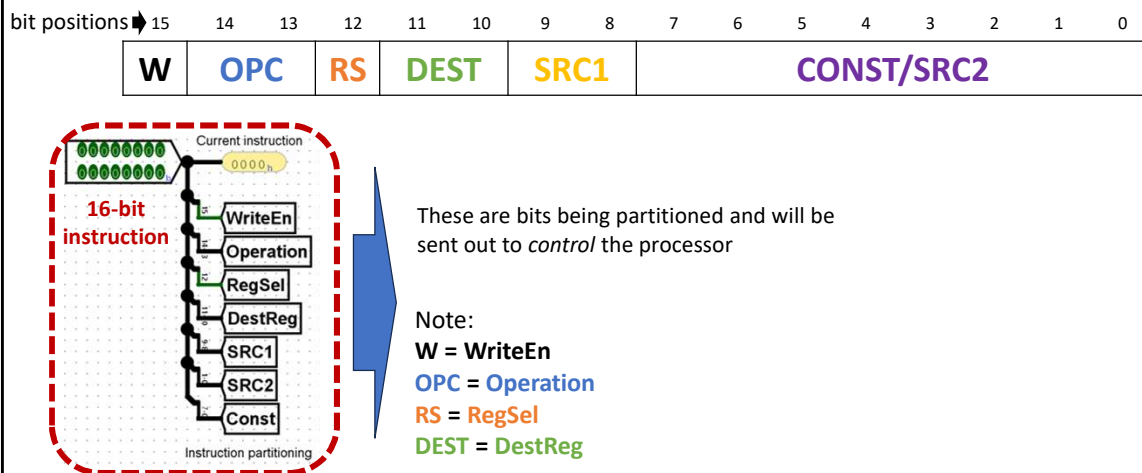
13

Now that we have mastered this instruction encoding and how instructions generally work, we are now ready to architect the processor. We will be doing it step-by-step.



Processor implementation: Step 1

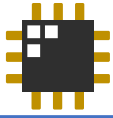
We first need to split the 16-bit instruction into parts



14

The first task is to separate the 16-bit instruction into its corresponding components. This is simply partitioning the entire 16 bits into parts, corresponding to the fields in the instruction encoding. I just used a more descriptive name for the connections:

- W = WriteEn (write enable)
- OPC = Operation (operation code)
- RS = RegSel (register select)
- DEST = DestReg (destination register)
- SRC1 = SRC1, still the same (source register 1)
- CONST/SRC2 = Const and SRC2, still the same (either the constant and the source register 2)



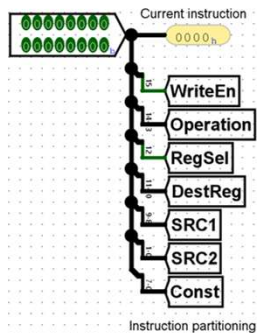
Processor implementation: Step 1

If we are operating using registers R0, R1, R2, R3...

We need to read their register values.

What are we missing?

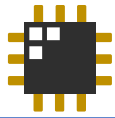
W	OPC	RS	DEST	SRC1	CONST/SRC2
---	-----	----	------	------	------------



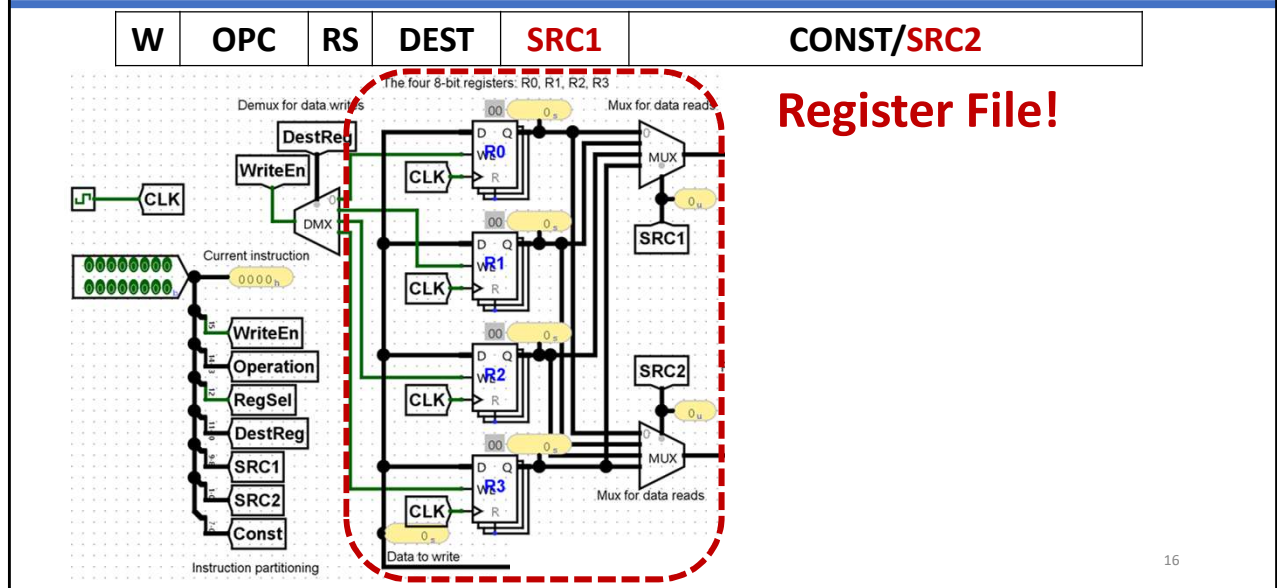
15

After partitioning, we can now start the operation. Whenever we execute the instruction, we want to get the values of the operands first before trying to operate on them.

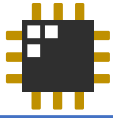
Since there are instructions that have both operands as registers, we have to get the values of those registers first. So, what are we missing?



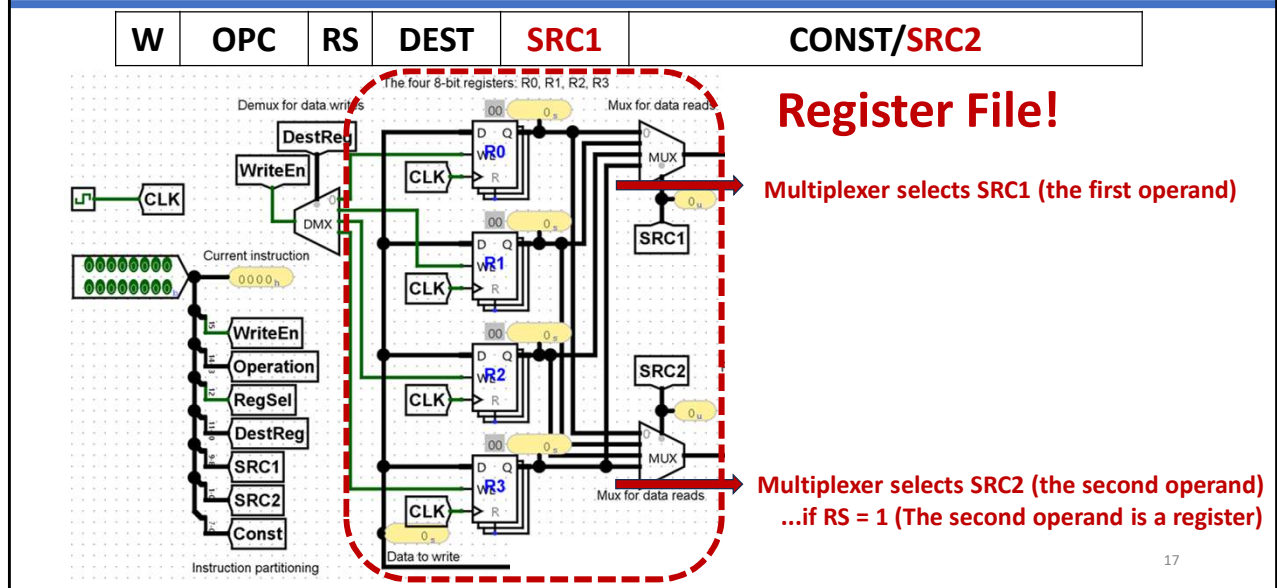
Processor implementation: Step 2



We need a register file for it. This is the same register file that was introduced in the previous lecture. This register file has 4 registers, exactly like the one in the previous lecture.



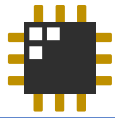
Processor implementation: Step 2



However, since we need to read the values of two registers at the same time, we add another multiplexer, essentially having the same functionality as the other multiplexer, to read one of the four register values. These two multiplexers work in parallel to output the values of the two operands we need for our instruction.

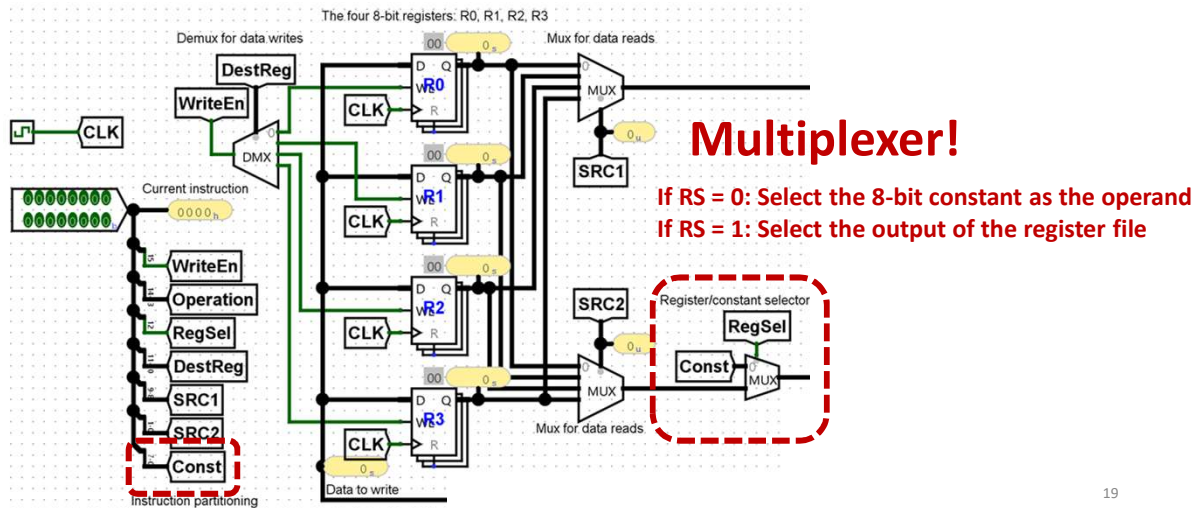


18



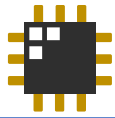
Processor implementation: Step 3

W	OPC	RS	DEST	SRC1	CONST/SRC2
---	-----	----	------	------	------------

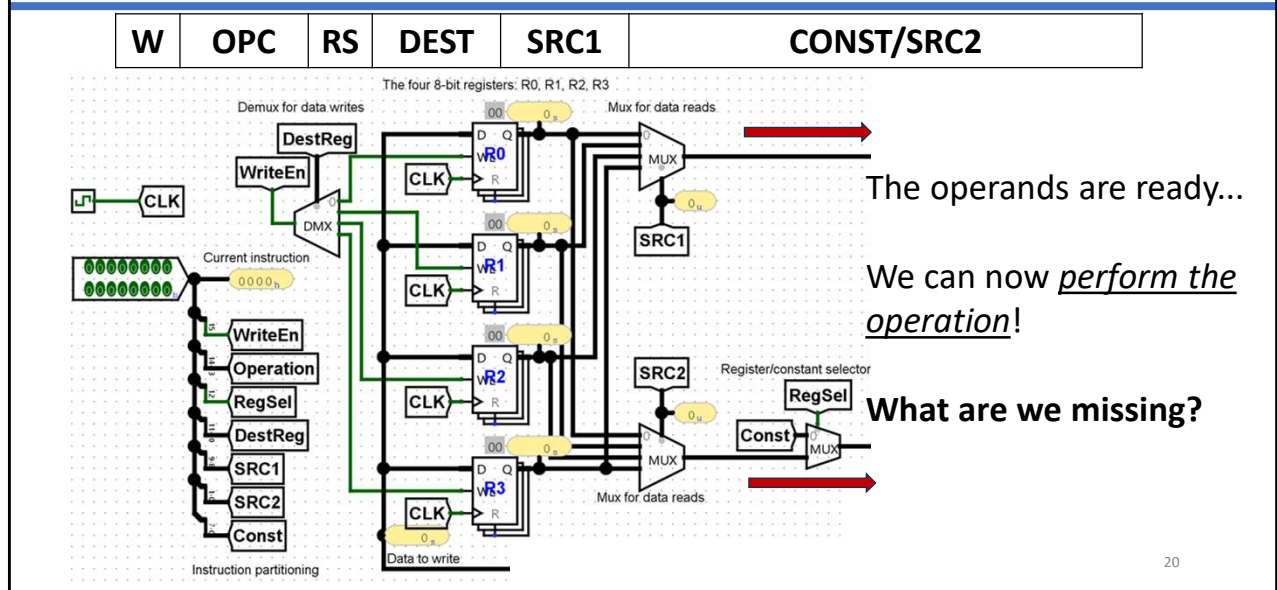


19

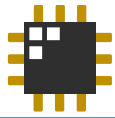
At this point, we have now read the values of the operands, we are now ready to perform the operation!



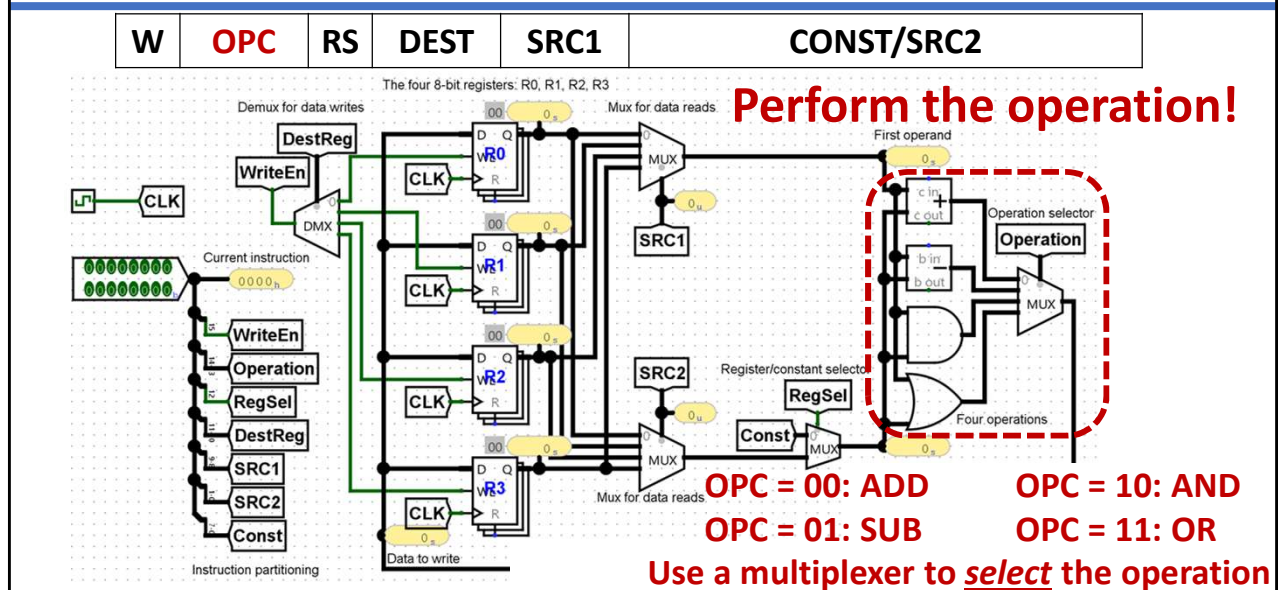
Processor implementation: Step 3



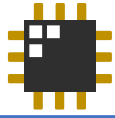
At this point, we have now read the values of the operands, we are now ready to perform the operation!



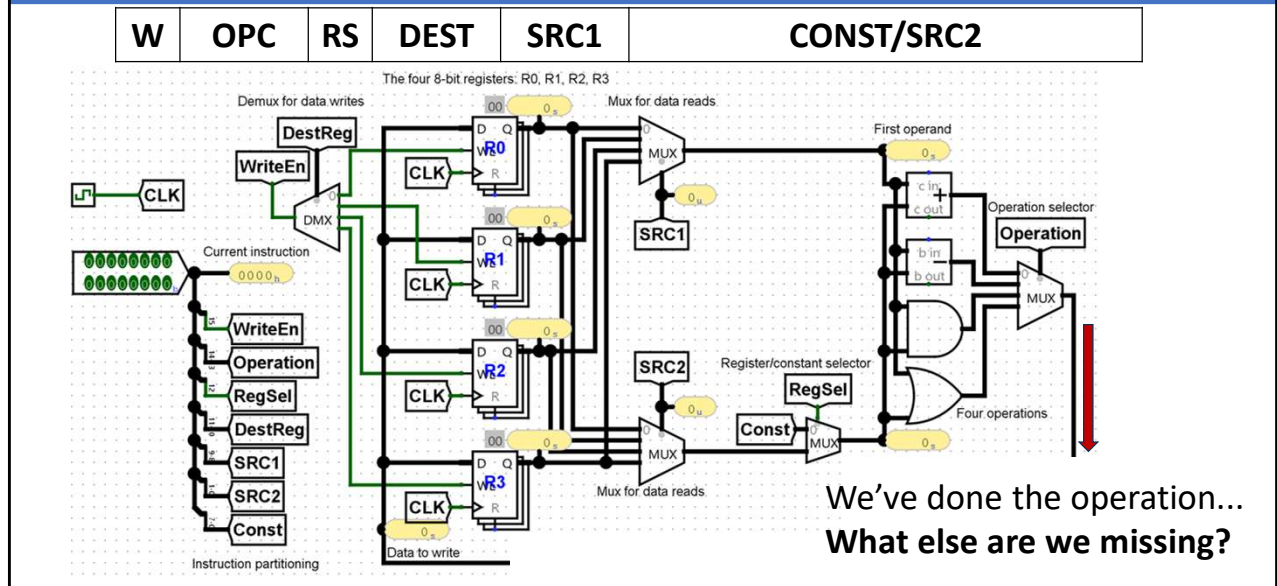
Processor implementation: Step 4



Depending on the OPC (Operation) field of the instruction encoding, we perform either one of the four operations: 00 for ADD, 01 for SUB, 10 for AND, and 11 for OR. This can be done by doing all four operations in parallel and then just selecting which of the four outputs will be selected as the actual result. This selection can be implemented by a 4-to-1 multiplexer and is selected by the OPC (Operation) bits.



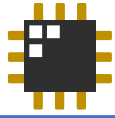
Processor implementation: Step 4



Once we have the result of the operation, we are now ready to update the register file!

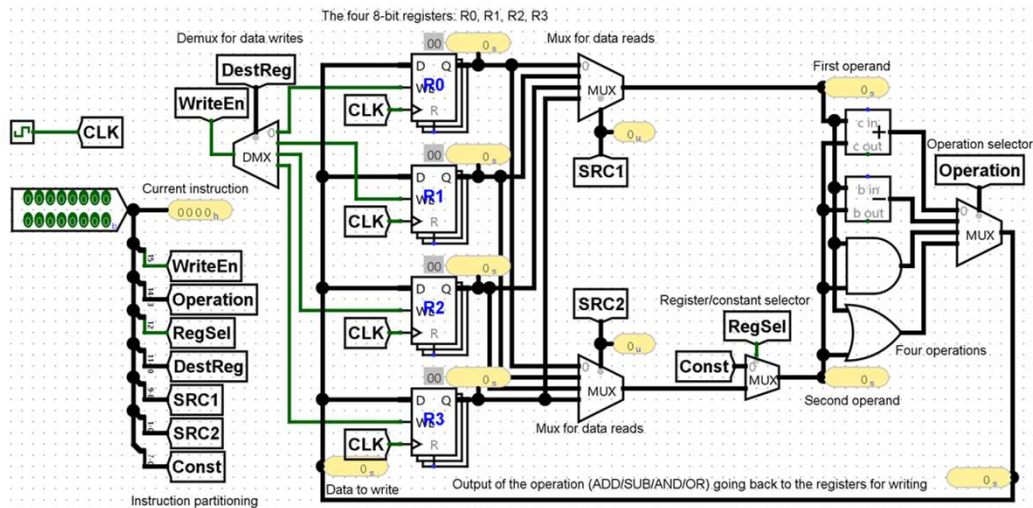


At this point, when we provide an instruction, the data will propagate through the register file, through the operations, and back to the register file for writing. The register file will update on the positive edge of the clock, as you've learned from the previous lecture.



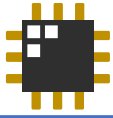
Processor implementation: Version 1

This processor can perform ADD/SUB/AND/OR on 4 registers and a constant



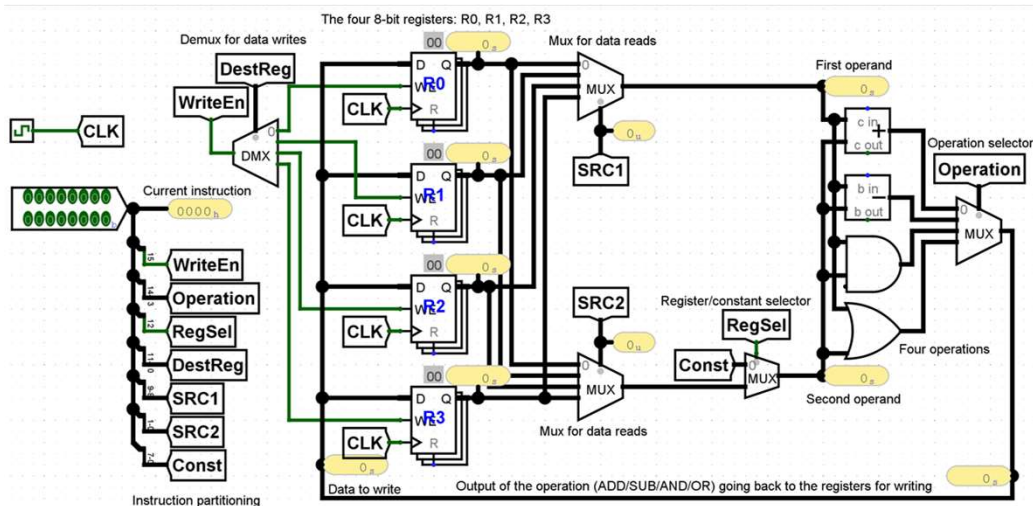
24

And that's it, we have our first version of the processor now. It can perform one of the four operations at any given time. It has four registers to choose from as its destination and source registers. It even supports a constant as the second operand. So far so good!

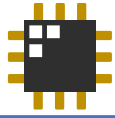


Processor implementation: Version 1

Try it out! SUBI R3, R1, -31 (0xADE1 in hex) ($R3 = R1 \text{ (initially 0)} - (-31) = 31$)

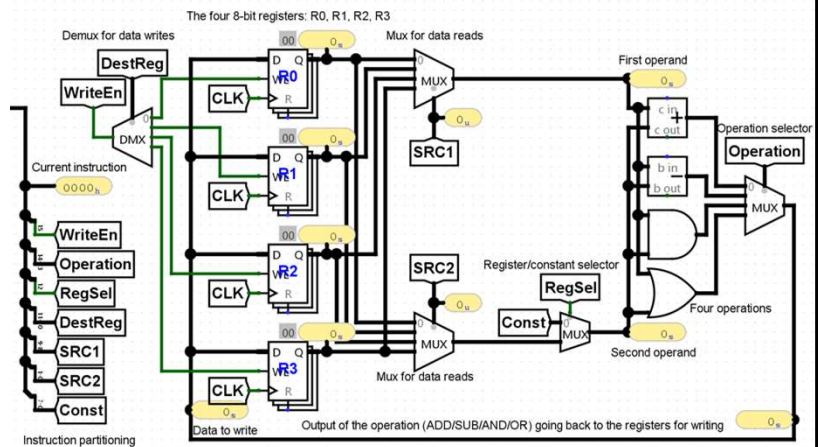


Here's a demonstration of the working processor (version 1). As you can see, the instruction is provided on the left-hand side corresponding to the SUBI R3, R1, -31. Different wires are probed to check what is happening on the processor. Since R1 is initially 0, the operation would become $0 - (-31) = 31$. The value of R1 and the selected constant value are probed together with the result of the operation in this demonstration. Indeed, the expected result of 31 appears at the output of the 4-to-1 multiplexer on the far right, ready to be written. After the positive edge of the clock arrives, R3 is updated to contain the value of 31, completing the instruction execution.

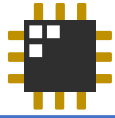


Running multiple instructions

- If we want to run consecutive instructions, we have to **store them in a memory**



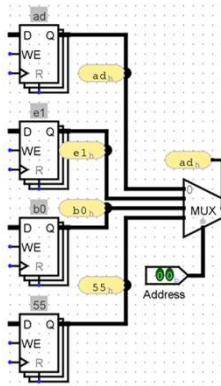
The previous demonstration manually inputted the instruction through an input port. This is not practical if we want to run several instructions one at a time. Thus, we have to store the instructions inside a memory so that we can just read the memory and have it provide the corresponding instructions, in machine code, every time.



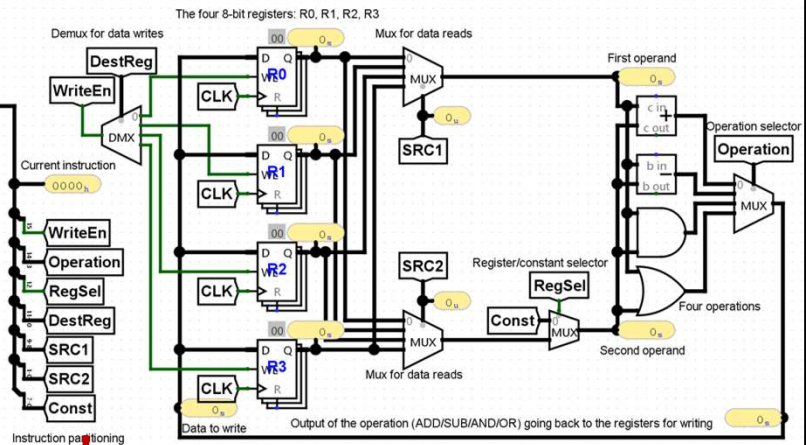
Running multiple instructions

- If we want to run consecutive instructions, we have to **store them in a memory**

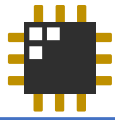
Recall from the previous lecture...



Something like this... but **bigger!**

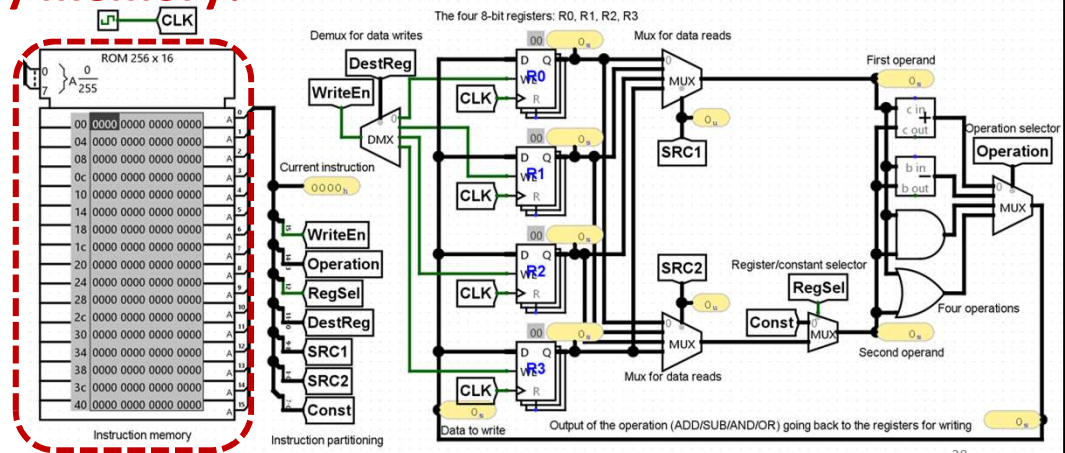


A memory structure has been presented in the previous lecture already, where different memory elements can contain different values and can be selected through a multiplexer. We would need something like that but on a much bigger scale to be able to handle more instructions.

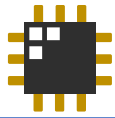


Processor implementation: Step 6

- If we want to run consecutive instructions, we have to **store them in a memory**
Read-only memory!



A read-only memory is added to our processor. The read-only memory architecture has already been presented in the previous lecture. You can imagine it as several registers containing some data and there is a multiplexer that is selecting which data should be treated as the final output. This read-only memory that you are seeing right now takes in an address and then depending on that address will output the corresponding data. The corresponding data is the instruction for the processor itself.



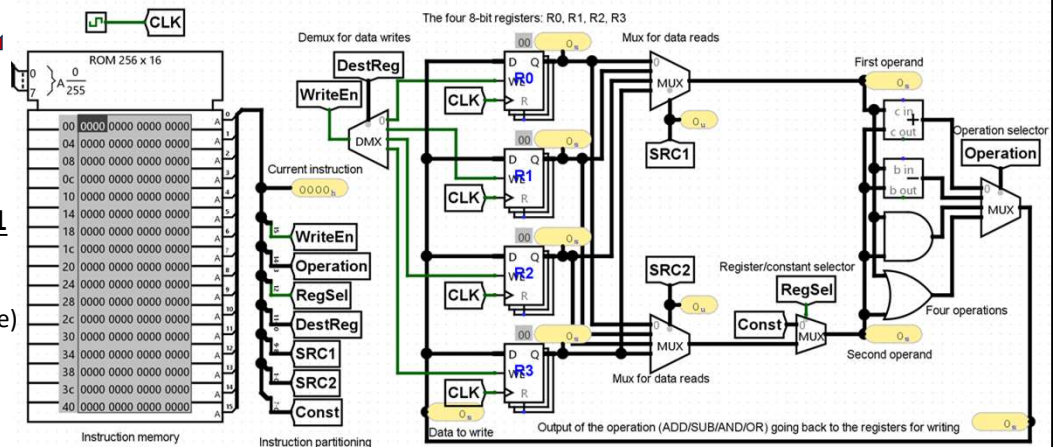
Processor implementation: Step 6

- If we want to run consecutive instructions, we have to **store them in a memory**

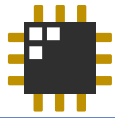
We have to provide an address.

Every cycle:
 $\text{Addr} = \text{Addr} + 1$
(we move to the next instruction every positive edge)

What are we missing?

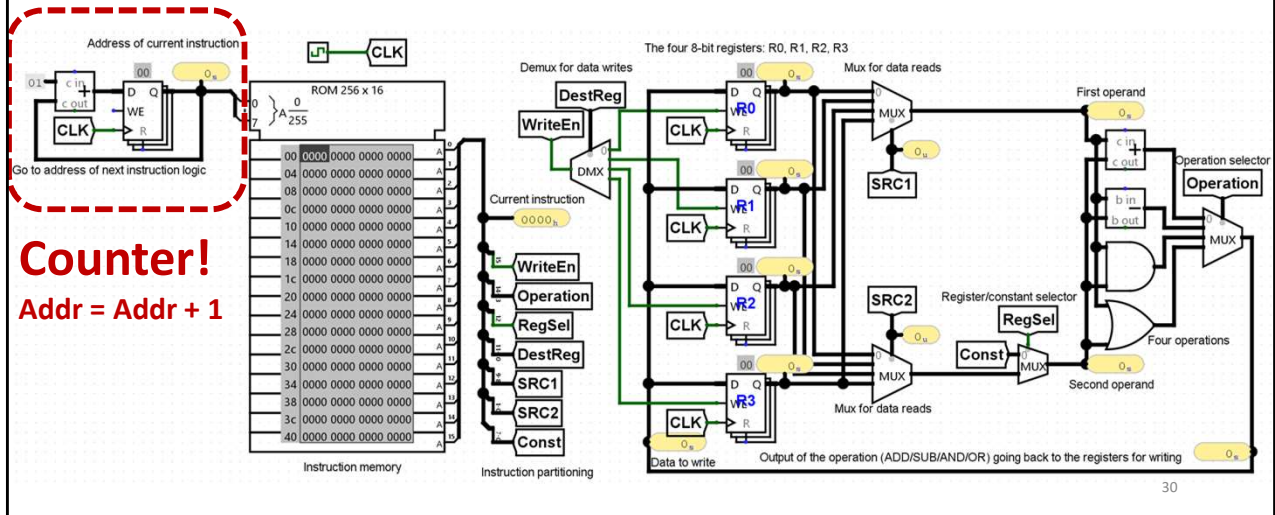


Whenever we finish executing an instruction, we should go to the next instruction, which should be located at the next address in the memory. The instruction always finishes after 1 positive edge of the clock. Therefore, the address of the memory should change to +1 every positive edge of the clock.

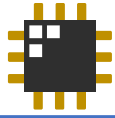


Processor implementation: Step 7

- If we want to run consecutive instructions, we have to **store them in a memory**

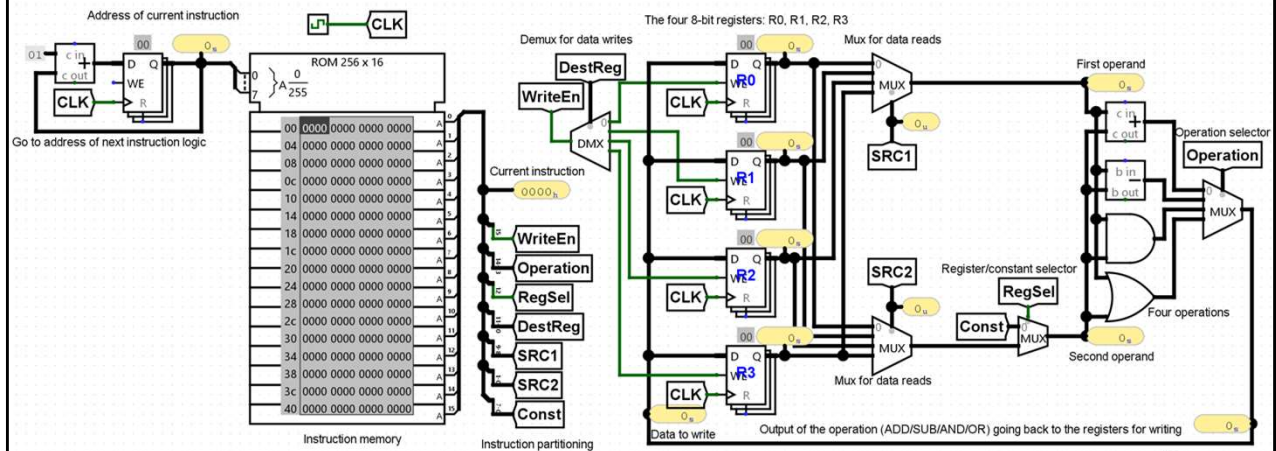


To implement this, we simply include a counter that adds 1 on every positive edge of the clock. This implementation of a counter has also been shown in the previous lecture. An adder is utilized to add a constant 1 to the current value of the count and will update every positive edge of the clock.

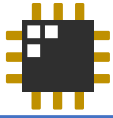


Processor implementation: Version 2

This processor can now run consecutive instructions!



That's it, with the inclusion of a read-only memory and a counter circuit to provide an address every clock cycle, the processor can now process instruction sequences! This is now version 2 of our processor.



Playing around with instructions

- Try running these instructions on the processor:
 - You have to convert them to machine code first

ADDI R3, R0, 113

ADDI R2, R2, 21

SUB R1, R3, R2

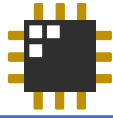
ANDI R1, R1, 15

OR R0, R2, R1

W	OPC	RS	DEST	SRC1	CONST/SRC2
---	-----	----	------	------	------------

32

To test our processor, we need a sequence of instructions to execute. We already know the process of converting an instruction to its corresponding machine code. However, it can get tedious.



Playing around with instructions

- Try running these instructions on the processor:

- You have to convert them to machine code first

ADDI R3, R0, 113

ADDI R2, R2, 21

SUB R1, R3, R2

ANDI R1, R1, 15

OR R0, R2, R1

```
1 def convert_to_two_complement(value, num_bits):
2     return bin((1 << num_bits) + value if value < 0 else value)[2:].zfill(num_bits)
3
4 def parse_instruction(instruction):
5     register_mapping = {'R0': '00', 'R1': '01', 'R2': '10', 'R3': '11'} # Register assignment for mov/demux selectors
6     operation_mapping = {'ADD': '00', 'SUB': '01', 'AND': '10', 'OR': '11'} # Operation assignment
7     condition_mapping = {'EQ': '00', 'NE': '01', 'LT': '10', 'GT': '11'} # Branch condition check for PC mux selector
8
9     parts = instruction.split()
10    write_en = str(int(not parts[0].startswith('B'))) # Always write unless it is a branch
11    operation = condition_mapping[parts[0].rstrip('.')] if parts[0].startswith('B') else operation_mapping[parts[0].rstrip('.')] # Extract the operation bits
12    const_sel = str(int(not parts[0].endswith('I') or parts[0].startswith('B'))) # Everything that ends with I is an immediate
13    destreg = '00' if parts[0].startswith('B') else register_mapping[parts[1].rstrip('.')] # This is a don't care if it is a branch instruction actually
14
15    # If instruction is branch, the source register is the register at index 1 (destreg), else it is at index 2 (since dest register would be at index 1)
16    srcreg = register_mapping[parts[1].rstrip('.')] if parts[0].startswith('B') else register_mapping[parts[2].rstrip('.')]
17
18    if parts[0].endswith('I'): # If the instruction is an immediate type, convert to two's complement notation
19        const = convert_to_two_complement(int(parts[3].rstrip('.')), 8)
20    elif parts[0].startswith('B'): # Branch instructions have their constants on index 2 instead of 3
21        const = convert_to_two_complement(int(parts[2].rstrip('.')), 8)
22    else: # If the instruction does not have a constant, then it is a purely register-based instruction
23        const = '000000' + register_mapping[parts[3].rstrip('.')]
24
25    binary_result = write_en + operation + const_sel + destreg + srcreg + const # Combine all binary into 16-bit instruction
26    hex_result = hex(int(binary_result, 2))[2:].zfill(4).upper() # Convert to hex
27    return hex_result
28 # End of actual assembler function, less than 30 lines!
```

You can write an assembler to speed up the process.

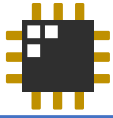
^ This is the Python **assembler** for our processor.

Given the defined instruction encoding, we should be able to automate the process!

W	OPC	RS	DEST	SRC1	CONST/SRC2
----------	------------	-----------	-------------	-------------	-------------------

33

We have provided an assembler for our processor, available in UVLE, to speed things up. You can use the provided Python-based assembler to play around with the processor that we are designing.



Playing around with instructions

- Try running these instructions on the processor:
 - You have to convert them to machine code first

ADDI R3, R0, 113 **0x8C71**

ADDI R2, R2, 21 **0x8A15**

SUB R1, R3, R2 **0xB702**

ANDI R1, R1, 15 **0xC50F**

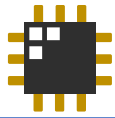
OR R0, R2, R1 **0xF201**

Given the defined instruction encoding, you can verify that our hexadecimal instruction codes are correct!

W	OPC	RS	DEST	SRC1	CONST/SRC2
---	-----	----	------	------	------------

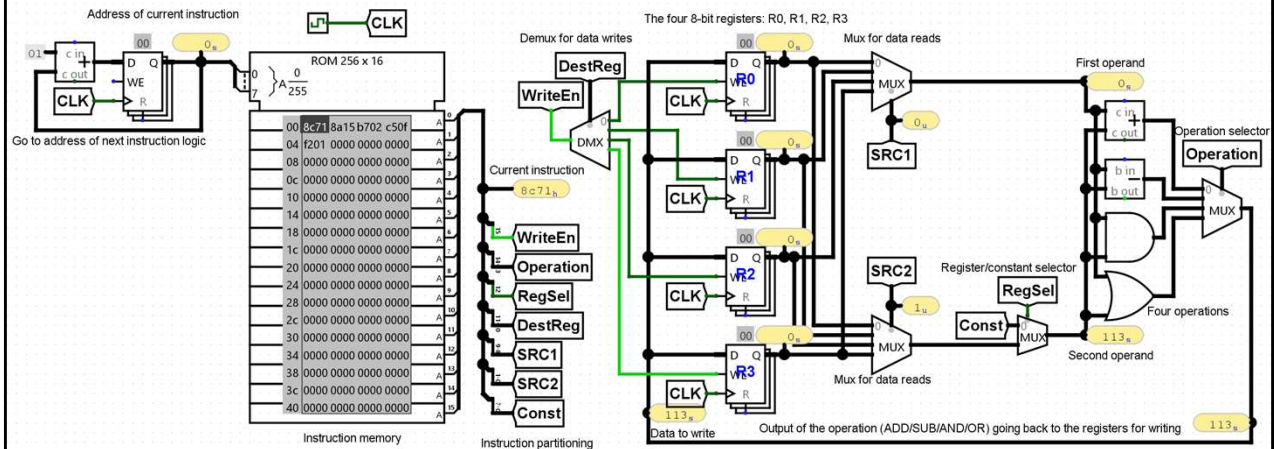
34

Converting the instructions to machine code will give us the following. You can either use the provided Python assembler, or you can manually do the conversion since the instruction encoding is known. Doing it manually can take time, but as long as you know what you are doing, the resulting machine code (currently written in hex) should be the same. Try it out on your own to be more comfortable with it!



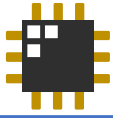
Processor implementation: Version 2

Try it out! Run all 5 instructions one at a time!



35

So now, loading those machine code instructions into our processor (version 2), we end up with this demonstration. The four instructions are in the memory. On every positive edge of the clock, the address moves to select the next instruction to execute. Every positive edge of the clock, the contents of the register update which corresponds to what the instruction wants to do. Everything is consistent so far. Our processor is evolving!



Supporting program loops

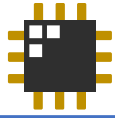
- Fun programs have loops!

```
r1 = 0
for r2 in range(5):
    r1 = r1 + 1
```

How do we do this on the processor?

36

Now, for the last improvement we want to incorporate into our processor. Typical programs have loops to make them more compact and efficient.



Supporting program loops

- Fun programs have loops!

ADDI R2, R2, 5

(R2 = 5)

loop:

ADDI R1, R1, 1

(R1 = R1 + 1)

SUBI R2, R2, 1

(R2 = R2 - 1)

If R2 != 0, go back to **loop**

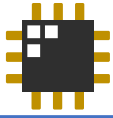
(how to do this?)

Note: The **loop** label is just a programming marker to where we want to go. It is not an instruction.
This one is *pointing* to the ADDI R1, R1, 1 instruction.

Instruction mastery questions:
How many times will we loop in this code?
What will be the final values of R1 and R2?

Shown on the slide is an example assembly code with a loop idea. It wants to keep on looping while the value of R2 is not equal to 0. Within the loop, R2 is being decremented by 1 at a time, so the loop will inevitably end after 5 iterations (since the value of R2 starts at 5 before entering the loop). R1, on the other hand, keeps on incrementing by 1. Thus, after all 5 iterations of the loop, R1 = 5.

But the question still remains, how do we jump from one point to another?



Supporting program loops

- **Branch*** instructions allow changing the current instruction address
 - You can add *conditions* on when to jump or not

ADDI R2, R2, 5 (**R2 = 5**)

loop:

ADDI R1, R1, 1 (**R1 = R1 + 1**)

SUBI R2, R2, 1 (**R2 = R2 – 1**)

BNE R2, -2 (**If R2 $\neq$ 0, go back 2 steps**)

EQ (Equal to 0)

NE (Not Equal to 0)

LT (Less Than 0)

GT (Greater Than 0)

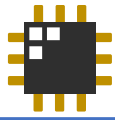
Branch is a computer architecture term. If you jump, you branch out of the normal program flow (one line after the other)

38

We will now have to support branch instructions. Branch instructions allow changing the current instruction address, thereby allowing us to go to any point in the code (effectively jump). If we jump backward, then we are creating a loop. Branch is a computer architecture term by the way, since it refers to branching out of the normal program flow. Branches have conditions on when to jump. If the condition is satisfied, we proceed to jump; otherwise we just proceed to the next instruction under it. For this example, we want to check for the condition of R2 not being equal to 0, so we use the condition code NE meaning not equal. We can make our processor support different condition codes for better flexibility when implementing more complex assembly programs. We can set four condition codes for now: EQ (equal to 0), NE (not equal to 0), LT (less than 0), or GT (greater than 0).

Also note how the branch instruction is written: BNE R2, -2. The register R2 appears there because it is the one that is being checked against 0 to see if it satisfies the condition NE. The -2 is a constant and refers to how many instructions will it jump to if the condition is satisfied. For this code, since we want to go back to the loop label (which points to the ADDI R1, R1, 1 instruction), that is 2 instructions back relative to the BNE instruction itself. Therefore, we put -2 as the constant (negative since we have to go back two

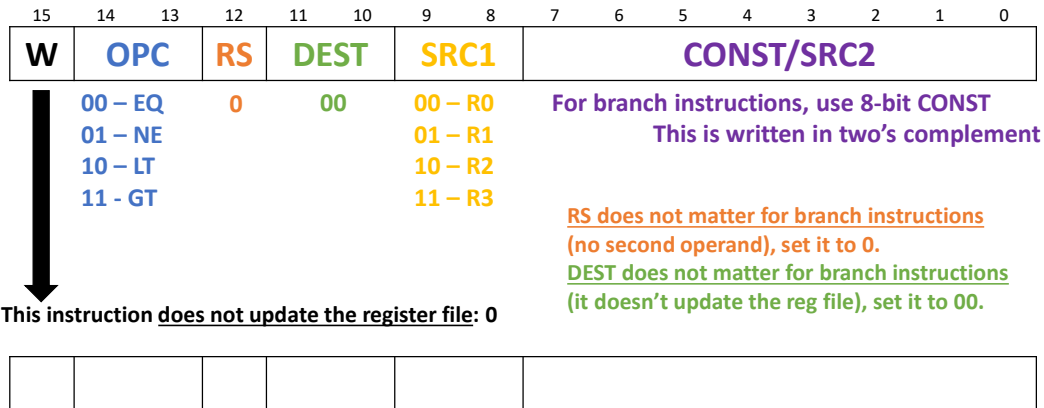
steps. The constant would be positive instead if we are jumping forward.



Supporting program loops

- How are branch instructions converted to machine code?

BNE R2, -2



39

So, how do we write the BNE instruction in machine code? We can follow the description of the instruction encoding fields to help us.

Since branch instructions do not need to update the register file (it only jumps, there's no operations being done), we set $W = 0$.

Since branch instructions perform comparison instead of an actual operation, we can repurpose the OPC field so that it represents the four conditions that we want to support: 00 for EQ (equal to 0), 01 for NE (not equal to 0), 10 for LT (less than 0), and 11 for GT (greater than 0). For BNE, we set $OPC = 01$.

Since branch instructions do not have a second operand (it only compares the first operand relative to 0), then RS is irrelevant. We set $RS = 0$.

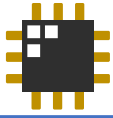
Since branch instructions do not update the register file (it only jumps!), then the destination register (DEST) is irrelevant. We set $DEST = 00$.

Branch instructions only use the SRC1 register as the main register to compare against 0 to check for conditions. For the instruction BNE R2, -2, the source register is R2. Therefore, we set $SRC1 = 10$ following the register mapping.

Finally, the last 8 bits are used to hold the constant, written in two's complement notation to support negative numbers. For the instruction BNE R2, -2, the constant is -2.

$2 = 0000\ 0010$. We want to get the negative representation so we do the two's

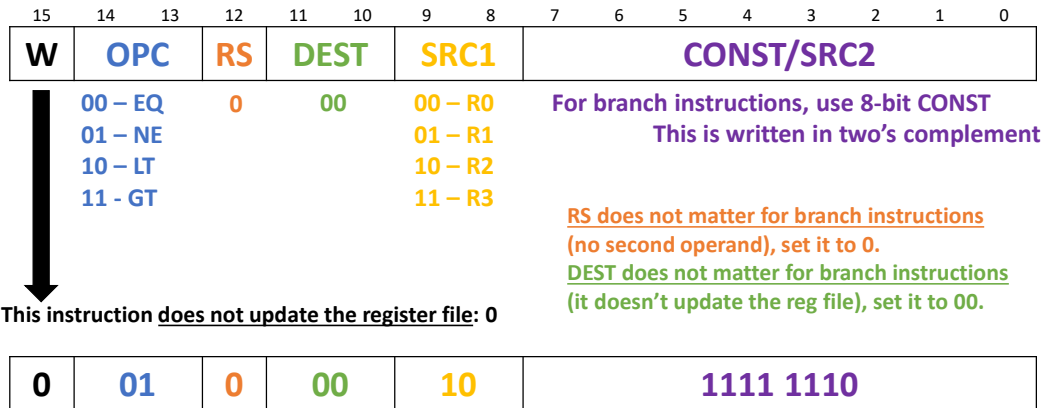
complement process. Inverting everything + 1 we get $1111\ 1101 + 1 = 1111\ 1110$. We set $CONST = 1111\ 1110$.



Supporting program loops

- How are branch instructions converted to machine code?

BNE R2, -2



40

Thus, we have the following:

W = 0 (not writing to register file)

OPC = 01 (NE)

RS = 0 (irrelevant)

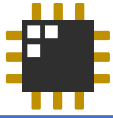
DEST = 00 (irrelevant)

SRC1 = 10 (R2)

CONST = 1111 1110 (-2 in two's complement)

0 01 0 00 10 1111 1110

0x22FE



Instruction encoding summary

- We're increasing in complexity! (***This is the last one, promise***)

W	OPC	RS	DEST	SRC1	CONST/SRC2
	If W = 0:		If W = 1:	00 – R0	If RS = 0: The second operand is 8-bit CONST This is written in two's complement
	00 – EQ		00 – R0	01 – R1	
	01 – NE		01 – R1	10 – R2	
	10 – LT		10 – R2	11 – R3	
	11 – GT		11 – R3		
	If W = 1:		If W = 0:		If RS = 1: Second operand is the last 2-bits SRC2
	00 – ADD		00 – branches do not write		0000 0000 – R0
	01 – SUB	If W = 0:			0000 0001 – R1
	10 – AND	0 – branches have no second operand			0000 0010 – R2
	11 – OR	If W = 1:			0000 0011 – R3
		0 – Constant second operand (ADDI/SUBI/ANDI/ORI)			
		1 – Register second operand (ADD/SUB/AND/OR)			

Sample inst

ADD R0,

SUBI R3,

BNE R2,

Sample instructions:

ADD R0, R2, R3
 SUBI R3, R1, -31
 BNE R2, -2

0: BEQ/BNE/BLT/BGT (Instructions that do not write to the register file)

1: ADD/SUB/AND/OR and ADDI/SUBI/ANDI/ORI (Instructions that write to the register file)

41

And with that, here's the updated instruction encoding that includes all the instructions that we have implemented so far.

W determines whether we write to the register file (1) or not (0). Only branch instructions don't write to the register file.

OPC changes depending if the instruction is a branch (00 for EQ, 01 for NE, 10 for LT, 11 for GT) or if the instruction is a typical arithmetic instruction (00 for ADD, 01 for SUB, 10 for AND, 11 for OR).

RS determines if the second operand is a constant (0) or a register (1). It is not relevant for branches (0).

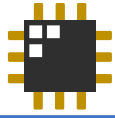
DEST determines the register mapping for the destination, the register that will be updated (00 for R0, 01 for R1, 10 for R2, 11 for R3). It is not relevant for branches (00).

SRC1 determines the register mapping of the first operand. It follows the same binary mapping for the register names.

CONST/SRC2 is treated as the 8-bit two's complement constant if RS = 0 or for branches. If RS = 1, then it maps the second register operand of the instruction (last 2 bits), padded with six 0s before it.

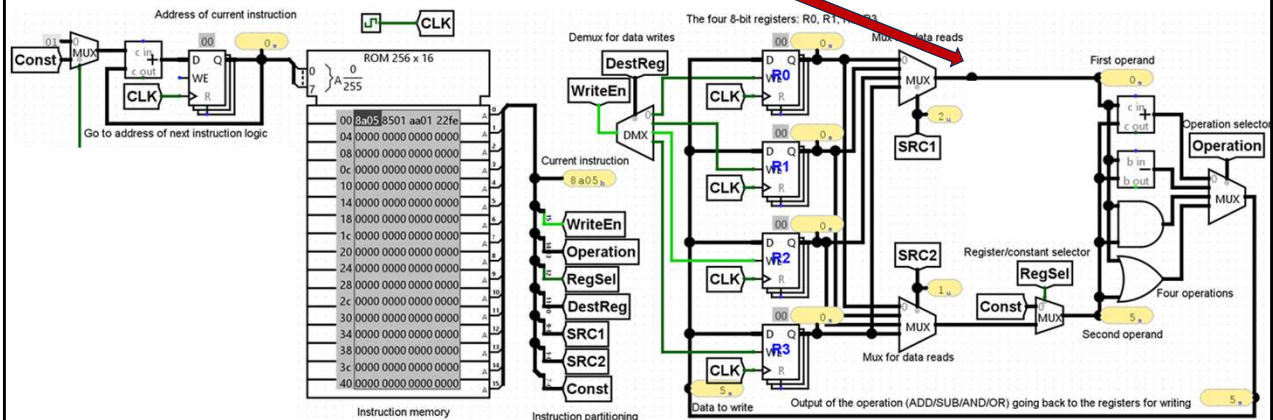
Some sample instructions are also written as a guide, with different fields

colored accordingly with the instruction encoding.



Processor implementation: Step 8

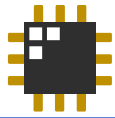
Branch instructions compare the SRC1 value (the first operand) relative to 0
What are we missing?



42

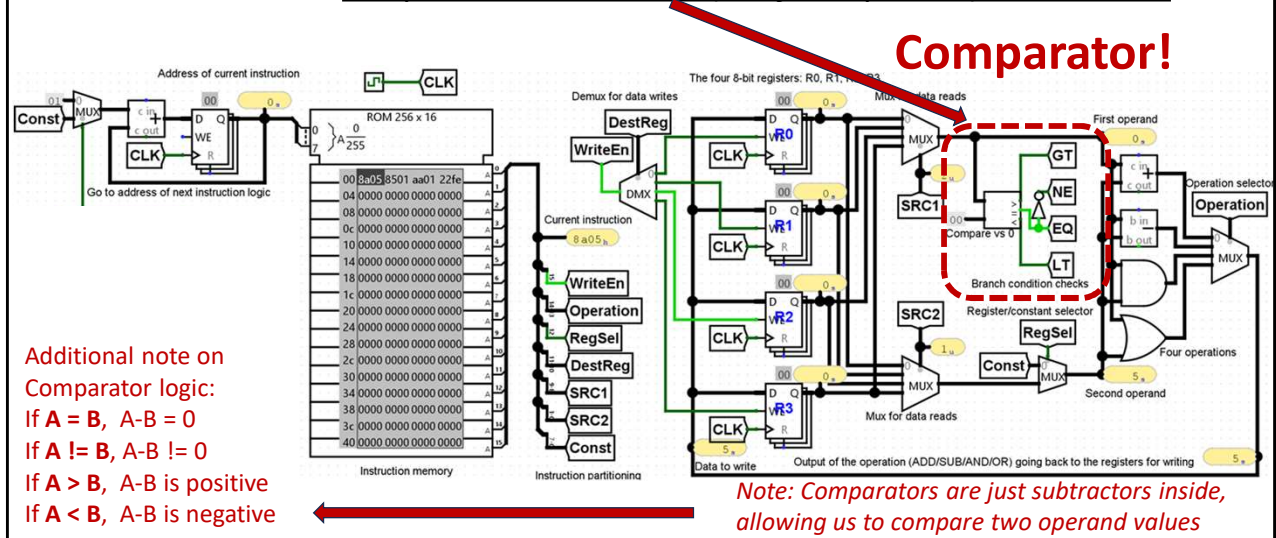
So now, after defining instruction encoding for branches, we have to modify the processor hardware to support it.

Branch instructions would always compare the value of the first operand (SRC1) relative to 0. This is how we determine the condition GT (greater than 0), NE (not equal to 0), EQ (equal to 0), and LT (less than 0).



Processor implementation: Step 8

Branch instructions compare the SRC1 value (the first operand) relative to 0



Since we are comparing one register relative to 0, we can use a comparator to do it. A comparator is a logic block that has a subtractor in it.

If we are comparing A and B:

We can say that $A = B$, if $A - B = 0$.

We can say that $A \neq B$, if $A - B \neq 0$.

We can say that $A > B$, if $A - B > 0$ (positive, MSB is 0 in two's complement)

We can say that $A < B$, if $A - B < 0$ (negative, MSB is 1 in two's complement)

We can use this comparator to compare the register value relative to 0 and then attach the corresponding conditions to the comparator outputs.

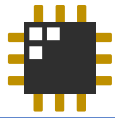
At this point, we also have to check for the OPC value to see if the condition matches what is being checked by the branch instruction.



We need to check if
we need to *branch out*
or not...

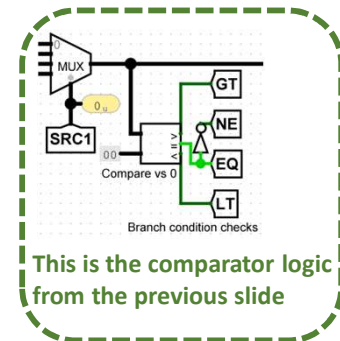
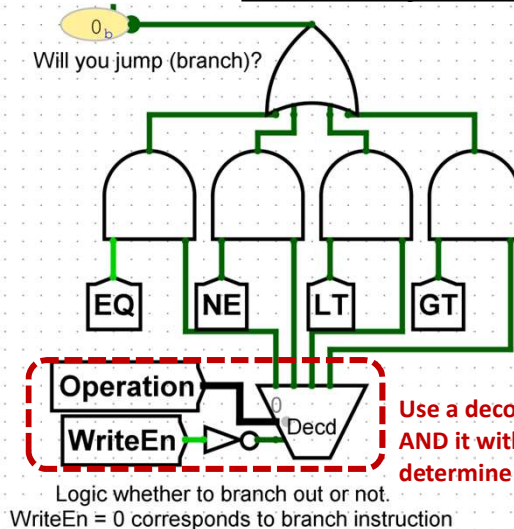
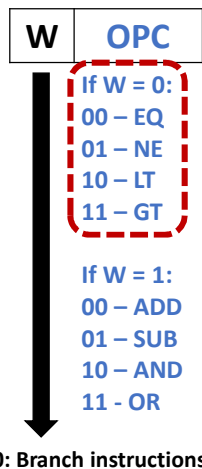
[illegible]

At this point, we also have to check for the OPC value to see if the condition matches what is being checked by the branch instruction.



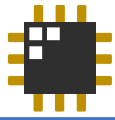
Processor implementation: Step 9

We check for the operation code to determine if the condition is satisfied



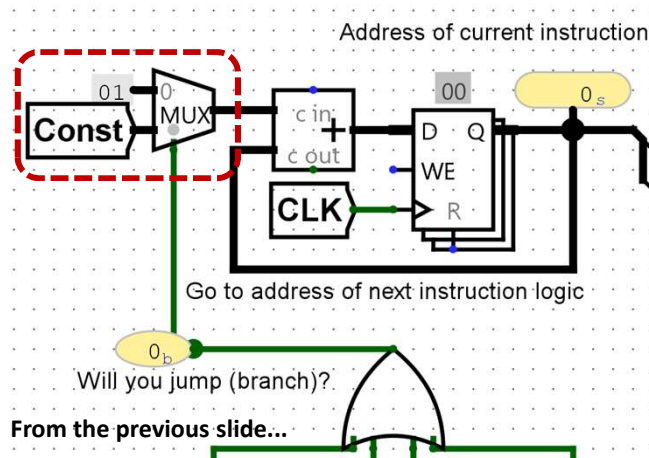
Use a decoder to identify the branch condition. AND it with the comparator outputs to determine if you're branching out or not.

We can do this by incorporating a decoder. If we pass the OPC (Operation) to the decoder, the decoder outputs one of the four signals which would correspond to the branch condition being checked. We can then AND these with the comparator outputs that we have integrated before. ORing all four of these signals together allows us to determine whether a branch condition is satisfied, which will then be a signal of whether we should jump or not. This decoder is then only enabled if W (WriteEn) = 0 (that's why there is a NOT gate in there before going to the decoder). This ensures that the branching logic is only active if we are executing a branch instruction (W = 0).



Processor implementation: Step 10

If the condition is satisfied, we have to branch out to a new address



We have to modify the address counter to add a constant instead of just +1 when branching out

Recall the example:

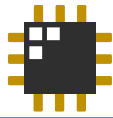
BNE R2, -2

This means that if the condition is true, our new address is -2 from where we are right now (-2 is a constant in the instruction)

46

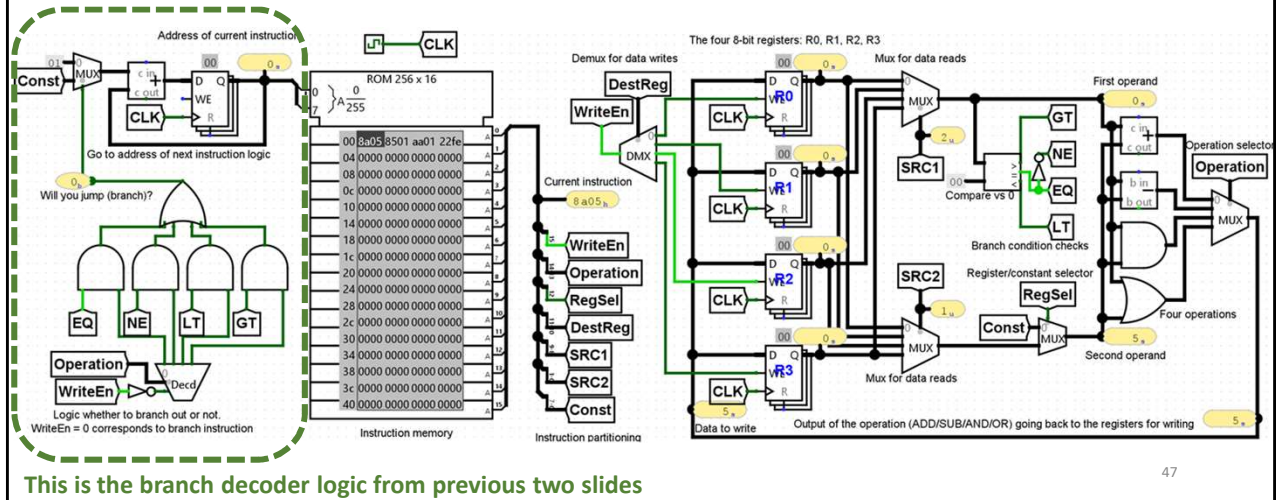
We will also have to modify the +1 counter we had earlier that generates our address. This modification is a variant of the up/down counter that we implemented in the previous lecture. By integrating a multiplexer on the counter, we can select whether to add 1 to the current address (normal execution flow) or add the constant to the address (if we are jumping). This enables the branching out process to happen!

You can recall how in the example instruction BNE R2, -2, we will have to jump to address -2 whenever the condition is true. So the -2 must be added to the address if we are to branch out. The -2 is a constant that was provided in the instruction, so we have to add this constant to the address to be able to branch out.

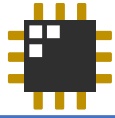


Processor implementation: Step 10

Branch instructions allow us to go to a different address if the condition is true

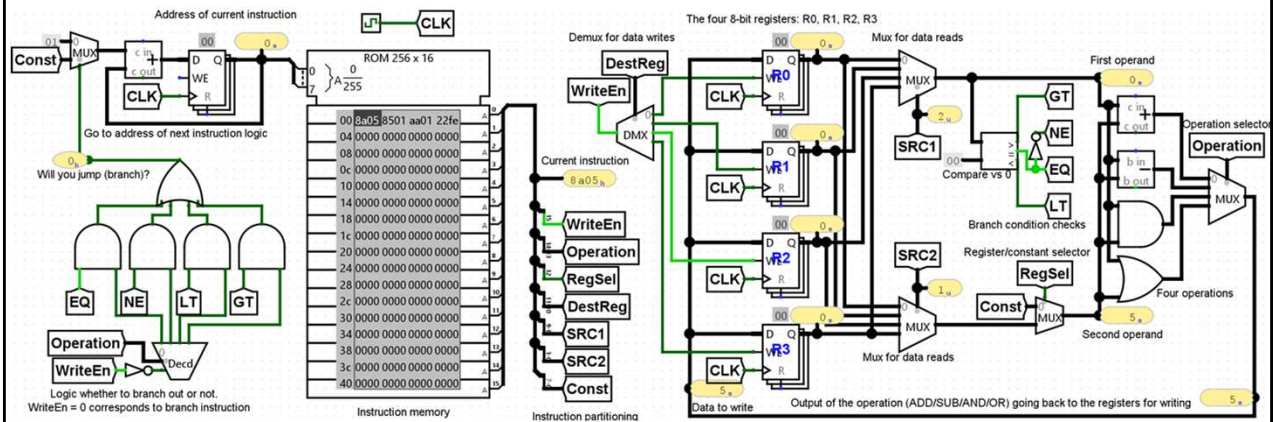


We integrate that decoder logic back into our processor. This allows us to jump to a different instruction address whenever the condition that is being checked by the instruction is true.

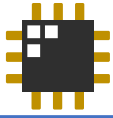


Processor implementation: Final Version!

This processor can now handle branch instructions as well!



And thus, we finally have our processor version 3, the final version. This processor now supports branch instructions as well!



Instruction encoding summary

- Recap: The instruction encoding that the processor supports

W	OPC	RS	DEST	SRC1	CONST/SRC2
↓	If W = 0:	↓	If W = 1:	00 – R0	If RS = 0: The second operand is 8-bit CONST This is written in two's complement
	00 – EQ		00 – R0	01 – R1	
	01 – NE		01 – R1	10 – R2	
	10 – LT		10 – R2	11 – R3	
	11 – GT		11 – R3		
			If W = 0:		If RS = 1: Second operand is the last 2-bits SRC2
	If W = 1:		00 – branches do not write		0000 0000 – R0
	00 – ADD				0000 0001 – R1
	01 – SUB				0000 0010 – R2
	10 – AND				0000 0011 – R3
	11 – OR				
		If W = 0:			
		0 – branches have no second operand			
		If W = 1:			
		0 – Constant second operand (ADDI/SUBI/ANDI/ORI)			
		1 – Register second operand (ADD/SUB/AND/OR)			

Sample instructions:

ADD R0, R2, R3

SUBI R3, R1, -31

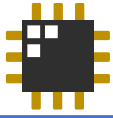
BNE R2, -2

0: BEQ/BNE/BLT/BGT (Instructions that do not write to the register file)

1: ADD/SUB/AND/OR and ADDI/SUBI/ANDI/ORI (Instructions that write to the register file)

49

This is just a recap of the final instruction encoding that the processor supports. Using this encoding, we can write more complex programs with loops now.



Sample program execution with loops!

- Let's run this now!

ADDI R2, R2, 5	0x8A05
loop:	<i>(this is just a label)</i>
ADDI R1, R1, 1	0x8501
SUBI R2, R2, 1	0xAA01
BNE R2, -2	0x22FE

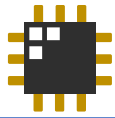
The **loop** label is only relevant when writing the instructions. It is just a programming marker to where we want to jump into if the branch condition is true. It has no machine code equivalent since it is not an instruction.

The **loop** label is technically marking where the ADDI R1, R1, 1 instruction is (the first instruction below it).

Relative to the BNE instruction, ADDI R1, R1, 1 is -2 instructions from it. (Negative since it goes back 2 steps) 50

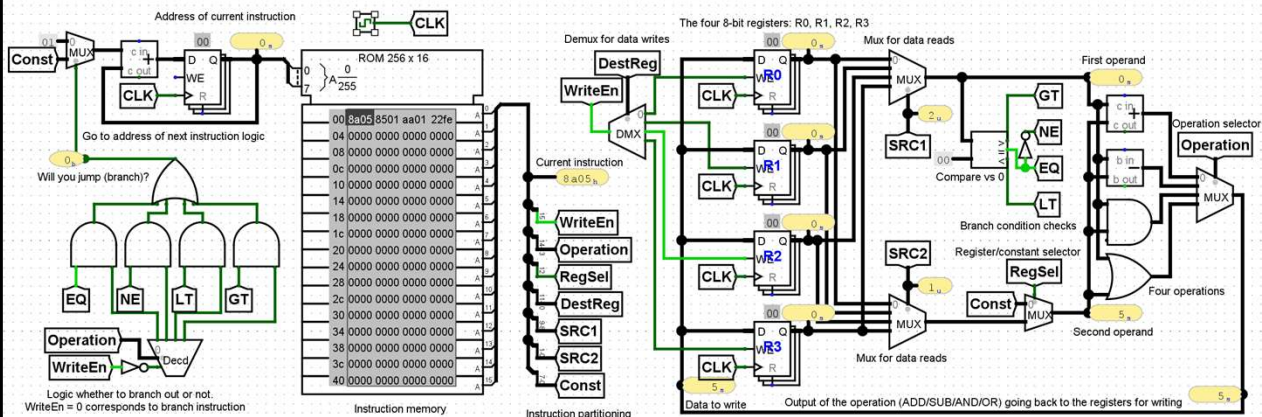
Let's go back to the initial assembly program that motivated us to implement the branch instruction. Here's the corresponding machine code of every instruction listed. Note that the 'loop' label does not have a corresponding machine code since it is not an instruction per se. It is just a marker when we are writing the assembly code to help us figure out how the program is run. However, what really happens is that we are just calculating how many instructions should we skip when branching. This is why BNE R2, -2 is written that way. -2 represents the number of instructions to skip. Negative means to go back and 2 means two instructions – so go back two instructions from where the BNE is. This is where the ADDI R1, R1, 1 instruction is at. This is where the 'loop' label is actually pointing at.

As always, you can try to manually convert the instructions to their corresponding machine code given the instruction encoding in the previous slide. Verify it on your own!



Sample program execution with loops!

• Let's run this now!

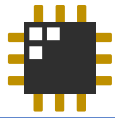


Note how the address goes back to '1' (Instruction 0x8501, which is the ADDI R1, R1, 1 instruction) after address '3' (Instruction 0x22fe, which is the BNE R2, -2 instruction). That's the loop in action!

Check the values of R1 and R2 as we go through the loop. Are they correct every step of the way?

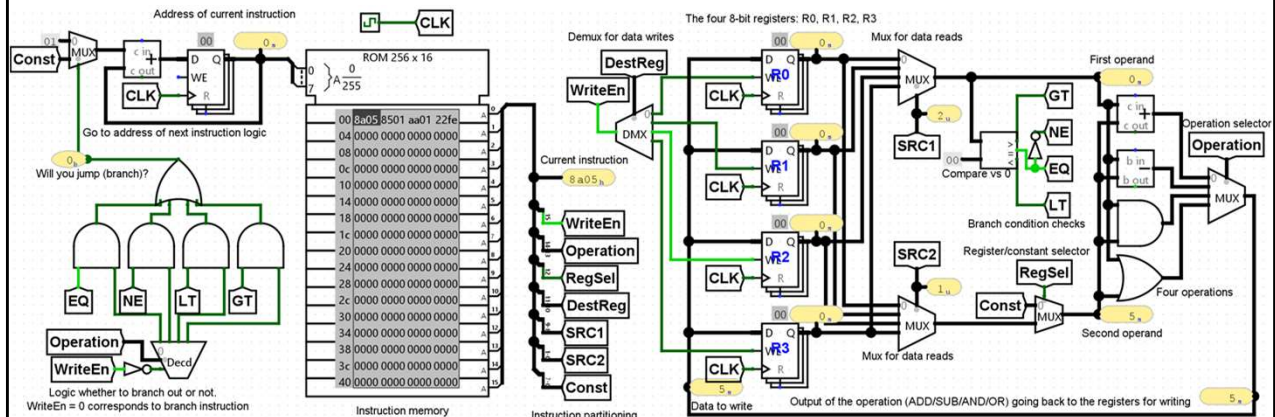
51

And so, running the processor (version 3) using the instructions in the previous slide, we have this demonstration. You can actually see the loop happening inside the read-only memory (since it highlights the current instruction being executed at any given time). We iterate the loop 5 times and we can see R1 and R2 changing values as we go through each iteration. R1 is being incremented by 1 every time, while R2 is being decremented by 1 every time. The loop will end once R2 reaches 0 since the BNE condition will fail. You can also see the processor about to execute instruction 0x0000 at address 4 when the loop ends since that is the next instruction after BNE when it fails the condition to jump. At this point, R1 will have a value of 5 since it got incremented 5 times.



Your first processor ☺

- 16-bit instructions, 8-bit data, 4 registers, 4 operations, 4 branch conditions



This is a *custom processor* made for EEE 113. Simple enough to be understood. Complex enough to implement simple programs. Previous EEE 113 offerings had 'Machine Architecture that is Really Intuitive and Easy' (MARIE) Now, for this offering, we have 'Architecture Design that's Easy to Learn' (ADEL) ☺

52

And that's it. We just implemented this processor! This processor has 16-bit instructions (recall the instruction encoding earlier). There are 4 registers in this processor and each register contains 8-bits each. It can support 8-bit constants as the second operand. It has four arithmetic-based operations: ADD/SUB/AND/OR. For branches, it supports up to 4 different conditions: EQ/NE/LT/GT.

We can implement simple programs using this processor and we will be exploring that in the next lecture.

Note that this processor is custom-made for EEE 113. This is being stated in case you tried searching for it online – you won't be able to. A lot of processors available online, while being advertised as simple, can still be relatively complex, especially given the timeframe we had in studying this. More complex processor architectures will be studied in EEE 153. For EEE 113, however, this processor that we have implemented is already simple enough to be understood but complex enough to run simple programs, which is what we will be doing next time.